In this chapter, you'll see:

- What kind of information RSpec stores about each spec
- How you can tag your examples with custom information
- How to perform expensive setup only when you need to
- How to run just a subset of your specs

CHAPTER 8

# Slicing and Dicing Specs with Metadata

Over the course of this book, you've followed a key principle that has made your specs faster, more reliable, and easier to use: *run just the code you need*. This principle shows up in several practices you've been using as you've followed along with the code examples:

- When you're isolating a failure, run just the failing example.
- When you're modifying a class, run just its unit tests.
- When you've got expensive setup code, only run it for the specs where you need it.

A key piece of RSpec that's made many of these practices possible is its powerful metadata system. Metadata undergirds many of RSpec's features, and RSpec exposes the same system for your use.

You've used metadata several times so far in your examples. In this chapter, we'll take a closer look into how it works. By the end, you'll understand enough to dictate exactly when and how your specs run.

## Defining Metadata

RSpec's metadata solves a very specific problem: *where do I keep information about the context my specs are running in?* By *context*, we mean things like:

- Example configuration (for example, marked as skipped or pending)
- Source code locations
- Status of the previous run
- How one example runs differently than others (for example, needing a web browser or a database)

Without some way of attaching data to examples, you (and the RSpec maintainers!) would be stuck juggling global variables and writing a bunch of bookkeeping code.